The American University in Cairo
Computer Science and Engineering Department

CSCE 239L/2302 Digital Design 1 Lab – Quiz – Fall 2025
Duration: 90 minutes, Total Marks: 10 marks

Examination Instructions:

- Do not start the exam until asked to do so.
- The exam is closed book. Opening your browser, checking your phone, or talking to any colleague will be considered cheating.
- The exam consists of **3** questions.
- Please zip all the problem deliverables by the end of the exam and upload it to Canvas under the assignment with the name "Quiz".
- **You can only ask the instructor/TA for help in your code if you get a synthesis error or warning that you do not understand. And you can do this <u>only once</u>.**
- Please save your project files continuously throughout the exam duration.

Exam deliverables:
One zipped folder containing:

1. barrel_rotator.v file
2. barrel_rotator_TB.v file and a clear screenshot of Tcl console output and the simulation waveform output
3. barrel_rotator.xdc constraint file and a photo of the FPGA board

Question 1 (4 pts):

Design and implement a **4-bit barrel rotator**, `barrel_rotator.v`, that can rotate its input bits **left or right** by a variable amount.

A **rotation** means that when bits are shifted out from one end, they wrap around and reappear at the other end.
<u>Example:</u> if input data_in = 1011, rotating left by 1 gives 0111, rotating left by 2 gives 1110, and so on.

The barrel rotator receives the following inputs:

- **4-bit `data_in`:** Data word to rotate
- **2-bit `sel`:** Rotation amount in bits (0–3)
- **1-bit `dir`:** Rotation direction, 0 for Left and 1 for Right

The outputs of your module should be:

- **4-bit `data_out`:** Rotated output

Implement the 4-bit barrel rotator in Verilog using behavioral and dataflow modeling.

Question 2 (4 pts):

Simulate `barrel_rotator.v` design module with a testbench module named `barrel_rotator_TB.v` using the following test cases:

Case	data_in	sel	dir	data_out	Explanation
1	1110	00	0	1110	No rotation
2	1110	01	0	1101	Rotate left by 1
3	1110	10	0	1011	Rotate left by 2
4	1110	11	0	....	Rotate left by 3
5	1101	01	1	1110	Rotate right by 1
6	1101	10	1	....	Rotate right by 2
7	1101	11	1	....	Rotate right by 2

This should be a **self-checking testbench** that makes checks over the results of the 7 test cases stated above and issues a *passed* or *failed* message over the Tcl console.
Attach a **clear screenshot of the console output** clearly showing whether each of the test cases passed or failed, and a **clear screenshot of the simulation waveform** clearly showing all test cases.

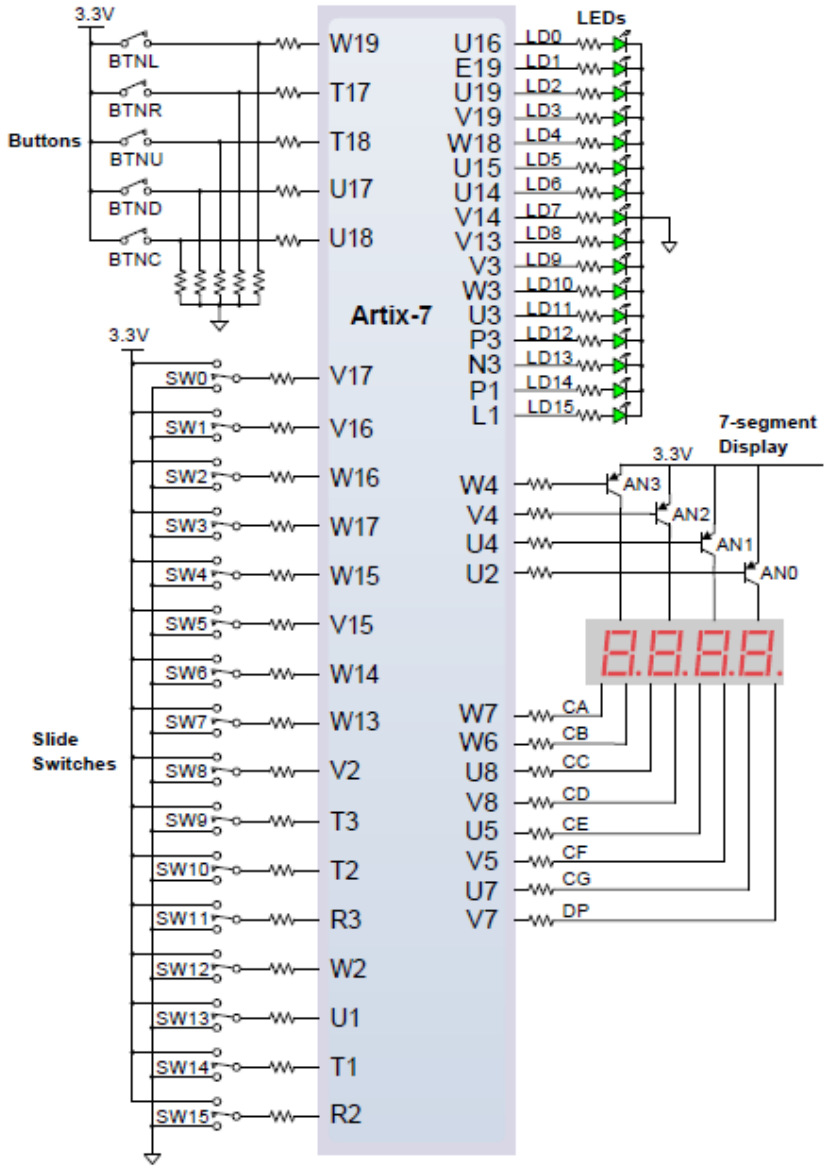
Question 3 (2 pts):

Write a constraint file named `barrel_rotator.xdc` for the `barrel_rotator.v` design module. Synthesize, Implement, generate bitstream and program your design on the Basys 3 board. Attach a **photo of the FPGA board** showing one of the test cases of Q2.

Good Luck

Reference Sheet

General purpose I/O devices on the Basys3 board:

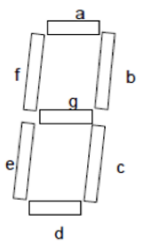


Constraint file syntax:

set_property PACKAGE_PIN <pin_number> [get_ports <signal>]
set_property IOSTANDARD LVCMOS33 [get_ports <same_signal>]

set_property PACKAGE_PIN <pin_number> [get_ports {<vector_element>}]
set_property IOSTANDARD LVCMOS33 [get_ports {same_vector_element}]

7-Segment digit



Testbench example:

Design module:

```
module simple_circuit(A,B,C,D,E);
     input A, B, C;
     output D, E;
     wire w1;

     assign D = ~C|(A&B);
     assign E = ~C;
endmodule
```

Test bench:

```
module simple_circuit_testbench ();

   reg A, B, C;
   wire x, y;

   // DUT module instantiation
   simple_circuit circ_sim(.A(A),.B(B),.C(C),.D(x),.E(y));

   // Stimuli Generator
   initial begin
      A = 1'b0; B = 1'b0; C = 1'b0;
      #100
      A = 1'b1; B = 1'b1; C = 1'b1;
      #100

      ..................................................
      ..................................................

   end
endmodule
```

Verilog Cheat Sheet

S Winberg and J Taylor

Comments

```
// One-liner
/* Multiple
   lines */
```

Numeric Constants

```
// The 8-bit decimal number 106:
8'b_0110_1010 // Binary
8'o_152       // Octal
8'd_106       // Decimal
8'h_6A        // Hexadecimal
"j"           // ASCII

78'bZ          // 78-bit high-impedance
```

Too short constants are padded with zeros on the left. Too long constants are truncated from the left.

Nets and Variables

```
wire [3:0]w; // Assign outside always blocks
reg  [1:7]r; // Assign inside always blocks
reg  [7:0]mem[31:0];

integer j; // Compile-time variable
genvar  k; // Generate variable
```

Parameters

```
parameter  N     = 8;
localparam State = 2'd3;
```

Assignments

```
assign Output = A * B;
assign {C, D} = {D[5:2], C[1:9], E};
```

Operators

```
// These are in order of precedence...
// Select
A[N] A[N:M]
// Reduction
&A ~&A |A ~|A ^A ~^A
// Compliment
!A ~A
// Unary
+A -A
// Concatenate
{A, ..., B}
// Replicate
{N{A}}
// Arithmetic
A*B A/B A%B
A+B A-B
// Shift
A<<B A>>B
// Relational
A>B A<B A>=B A<=B
A==B A!=B
// Bit-wise
A&B
A^B A~^B
A|B
// Logical
A&&B
A||B
// Conditional
A ? B : C
```

Module

```
module MyModule
#(parameter N = 8) // Optional parameter
 (input  Reset, Clk,
  output [N-1:0]Output);
 // Module implementation
endmodule
```

Module Instantiation

```
// Override default parameter: setting N = 13
MyModule #(13) MyModule1(Reset, Clk, Result);
```

Case

```
always @(*) begin
 case(Mux)
  2'd0: A = 8'd9;
  2'd1,
  2'd3: A = 8'd103;
  2'd2: A = 8'd2;
  default:;
 endcase
end

always @(*) begin
 casex(Decoded)
  4'b1xxx: Encoded = 2'd0;
  4'b01xx: Encoded = 2'd1;
  4'b001x: Encoded = 2'd2;
  4'b0001: Encoded = 2'd3;
  default: Encoded = 2'd0;
 endcase
end
```

Synchronous

```
always @(posedge Clk) begin
 if(Reset) B <= 0;
 else      B <= B + 1'b1;
end
```

Loop

```
always @(*) begin
 Count = 0;
 for(j = 0; j < 8; j = j+1)
  Count = Count + Input[j];
end
```

Function

```
function [6:0]F;
 input [3:0]A;
 input [2:0]B;
 begin
  F = {A+1'b1, B+2'd2};
 end
endfunction
```

Generate

```
genvar j;
wire [12:0]Output[19:0];

generate
 for(j = 0; j < 20; j = j+1)
 begin: Gen_Modules
  MyModule #(13) MyModule_Instance(
   Reset, Clk,
   Output[j]
  );
 end
endgenerate
```

State Machine

```
reg   [1:0]State;
localparam Start = 2'b00;
localparam Idle  = 2'b01;
localparam Work  = 2'b11;
localparam Done  = 2'b10;

reg tReset;

always @(posedge Clk) begin
 tReset <= Reset;

 if(tReset) begin
  State <= Start;

 end else begin
  case(State)
   Start: begin
    State <= Idle;
   end
   Idle: begin
    State <= Work;
   end
   Work: begin
    State <= Done;
   end
   Done: begin
    State <= Idle;
   end
   default:;
  endcase
 end
end
```